SOFTWARE ENGINEERING

LAB EXPERIMENT 2

Selecting a Software Process Model

A project-specific analysis for `kvstore`

Project	`kvstore`: A Distributed, Fault-Tolerant, Transactional Key-Value Store with an Integrated Operational Intelligence Layer
Submitted by	Neha Nupur
Programme	Master of Computer Applications (MCA)
Class / Section	MCA II Year
Roll Number	20251651064
Academic Session	2026

Experiment type: decision-making and process-model selection. No programming implementation is required.

1 Aim

To compare five software process models and select the most appropriate model for different project situations, with a detailed, evidence-based recommendation for the `kvstore` project.

2 Objectives

- Identify project characteristics that influence process-model selection.
- Compare Waterfall, Incremental, Prototyping, Spiral, and Agile against requirement stability, early delivery, customer feedback, and risk handling.
- Analyse all four cases specified in the Lab 2 handout.
- Apply the decision method to `kvstore` using its SRS and architecture.
- Justify the recommendation and record the result.

3 Project Profile: `kvstore`

The SRS describes `kvstore` as a distributed, replicated, transactional key-value store implemented in Go using only the Go standard library. It supports Put, Get, Delete, and atomic transactions; Raft-based replication and leader election; MVCC snapshot isolation and conflict detection; a checksummed write-ahead log (WAL); hybrid logical clocks; optional TLS/mutual TLS; metrics and a live dashboard; an operational-intelligence layer; sharding; and a two-phase-commit path for supported cross-shard writes/deletes.

The system is technically complex because correctness must hold across node failures, network delays/partitions, concurrent transactions, durable recovery, and distributed ordering. Its SRS documents testability expectations, a fault-injection harness, known limitations, and requirement traceability. These characteristics make risk control and continuous verification central to process selection.

4 Process Model Comparison

Factor	Waterfall	Incremental	Prototype	Spiral	Agile
Requirements	Stable and well understood before development begins.	Partly stable; features can be prioritised and delivered in stages.	Initially unclear; requirements are refined through prototype evaluation.	May evolve; risks and requirements are revisited during each cycle.	Expected to change through continuous stakeholder feedback.
Early delivery	Usually late, after major development phases are completed.	Working software is delivered incrementally.	An early sample or prototype is created for evaluation.	Working solutions may be delivered after each risk-driven cycle.	Frequent working releases are delivered in short iterations.
Feedback	Limited after requirements are formally approved.	Feedback is collected during reviews of each increment.	High feedback while validating requirements and proposed solutions.	Feedback is obtained during reviews of each development cycle.	Very high; feedback is continuous and informs priorities.
Risk handling	Risks are mainly identified and planned for at the beginning.	Risks are reduced progressively as increments are developed.	Useful for reducing uncertainty about requirements and design.	Very strong; explicit risk identification and analysis guide each cycle.	Risks are managed iteratively through collaboration and continuous testing.
Testing	Testing may occur relatively late in the development lifecycle.	Each increment is integrated and tested before further development.	The prototype is evaluated; the final system still requires complete testing.	Testing and risk evaluation are performed during every cycle.	Continuous testing and automated regression checks are encouraged.
Best fit	Predictable projects with stable requirements and limited expected changes.	Large systems that can be divided into manageable functional increments.	Projects with unclear user interfaces, workflows, or requirements.	Complex, technically uncertain, and high-risk systems.	Projects requiring frequent releases, changing priorities, and close collaboration.

Observation: No single process model is universally best. Selection depends on requirement stability, delivery expectations, feedback needs, technical uncertainty, and the potential cost of failure.

5 Case Analyses from the Lab 2 Handout

5.1 Case 1: College Payroll System

Situation: Salary rules, allowances, and deductions are known; major changes are not expected.

Requirements: Stable and documentable.

Risk: Comparatively low to moderate if payroll rules and statutory requirements are validated early.

Frequent feedback: Not essential, but payroll/HR should review requirements and calculations at planned checkpoints.

Selected model: Waterfall.

Requirements can be approved up front, and calculation rules can be traced to test cases. A formal sequence of requirements, design, implementation, testing, and deployment is appropriate. Payroll accuracy still requires thorough validation.

5.2 Case 2: New Food-Delivery App

Situation: The startup wants frequent releases and users may request features after trying the app.

Requirements: Evolving.

Early working software: Essential to evaluate ordering, payment, delivery tracking, and restaurant workflows.

Customer feedback: Very high and continuous.

Selected model: Agile.

Short iterations allow the team to release a minimum viable feature set, learn from real users, and reprioritise the backlog. Frequent integration and automated regression tests help protect existing flows.

5.3 Case 3: Hospital Emergency Monitoring System

Situation: Failure can have serious consequences; technical and safety risks must be studied carefully.

Risk: Very high.

System: Safety-critical and availability-sensitive.

Risk analysis: Must be explicit, repeated, documented, and linked to mitigations and tests.

Selected model: Spiral.

Each cycle identifies objectives and alternatives, evaluates risks, builds and verifies the selected solution, and plans the next cycle. A real medical system would also require applicable regulatory, clinical, and safety- engineering processes beyond this classroom comparison.

5.4 Case 4: New College Portal Interface

Situation: Students and staff are unsure how screens should look and want samples before finalising requirements.

Requirements: User-interface requirements are unclear.

Feedback: High, especially on navigation, terminology, accessibility, and task completion.

Selected model: Prototyping.

A low-cost interactive mock-up lets stakeholders react to concrete screens before detailed requirements are fixed. Feedback refines the specification. The prototype should be treated as exploratory unless engineered to production quality.

6 Applying the Decision to `kvstore`

6.1 Characteristics that drive the decision

Characteristic	Evidence and implication
Requirement stability	Core operations are clear, but detailed behaviour, edge cases, operational intelligence, sharding, and recovery need iterative refinement.
Technical complexity	High: Raft consensus, MVCC, WAL durability, HLC ordering, network transport, TLS, sharding, and transaction coordination interact.
Failure impact	High correctness impact: lost or inconsistently committed data, incorrect transaction outcomes, or unsafe recovery undermine the store's purpose.
Early validation	High: distributed behaviour and failure scenarios should be tested in small increments, not deferred to the end.
Feedback source	Developer tests, fault-injection results, metrics, dashboard behaviour, and evaluator/maintainer review provide feedback.
Architecture	Modular, layered Go packages with acyclic dependencies and a standard-library-only constraint support incremental implementation and testing.
Verification	SRS defines fault injection, crash-replay verification, race-detector checks, build/vet gates, and acceptance criteria.

6.2 Recommended model: Spiral with incremental, Agile-style delivery

The most suitable primary model is the **Spiral Model**, operationalised through small, testable increments and frequent review. This is a recommendation based on the risk profile, not a claim that the repository formally declares that it historically followed a named lifecycle.

1. **Risk is the dominant factor.**

   A distributed transactional store must handle leader changes, quorum decisions, concurrent writes, durable logging, crashes, and network faults. Spiral explicitly centres risk identification and mitigation in every cycle.

2. **Architecture can be validated in stages.**

   Build and validate a local transaction/storage path, WAL durability and recovery, consensus and replication, MVCC conflict handling, then transport security, observability, operational intelligence, and sharding. Each increment needs measurable acceptance tests.

3. **Feedback and evidence are continuous.**

   The SRS describes fault-injection tests, crash-replay verification, race-detector checks, build/vet acceptance gates, and documented limitations. These fit repeated risk-review cycles better than a single sequential pass.

Why not the alternatives as the primary model?

- **Waterfall:** Useful for traceability and formal reviews, but late discovery of consensus or recovery defects could force expensive redesign.
- **Incremental:** A strong delivery technique and part of the recommendation, but it does not emphasise explicit risk analysis as strongly as Spiral.
- **Prototyping:** Useful for dashboard or natural-language query experiments, but insufficient as the overall lifecycle for durable distributed transactions.
- **Agile:** Useful for prioritisation, short feedback loops, and continuous integration; Agile practices can be used inside Spiral cycles, while Spiral makes high-risk analysis explicit.

6.3 Suggested risk-driven cycle

Cycle	Objective	Risk question	Exit evidence
1. Foundations	Requirements, architecture, invariants, test strategy	Are guarantees and boundaries unambiguous?	Reviewed SRS, module boundaries, traceability, test plan
2. Durable core	Put/Get/Delete, transactions, WAL, recovery	Can acknowledged writes survive a crash?	Checksum, replay, crash-recovery tests
3. Consensus	Leader election, replication, quorum commit	Can nodes agree safely during failures?	Election, replication, partition, restart tests
4. Isolation	MVCC snapshots and conflict detection	Are concurrent transactions deterministic?	Snapshot isolation and first-committer-wins tests
5. Operations/security	TLS, metrics, dashboard, operational intelligence	Can operators detect issues without unsafe actions?	API/UI checks, TLS tests, metrics and bounded-query tests
6. Scale/release	Shard routing and supported cross-shard coordination	Can coordinator failure leave ambiguous outcomes?	Routing determinism and coordinator crash-recovery tests

7 Practical Process Controls

- **Plan:** Select a small, testable objective and identify affected SRS requirements.
- **Analyse risk:** List failure modes, impact, likelihood, and mitigation; prioritise data loss, inconsistent commits, split-brain behaviour, and unrecoverable logs.
- **Design:** Preserve the modular dependency hierarchy and define interfaces before integration.
- **Implement:** Deliver one vertical slice or module increment at a time.
- **Verify:** Run focused unit/integration tests, fault-injection and crash-recovery tests, race detection, build, and vet checks as appropriate.
- **Review:** Inspect evidence, metrics, unresolved defects, and documented limitations before accepting an increment.
- **Re-plan:** Update priorities based on evidence; an untested capability must not be treated as verified.

8 Decision Matrix for `kvstore`

Ratings are qualitative judgements for this project, not measurements. A score of 1 is a weak fit and 5 is a strong fit.

Criterion	Weight	Waterfall	Incremental	Prototype	Spiral	Agile
Requirement evolution	15%	4	4	5	4	5
Risk analysis	30%	2	3	2	5	3
Early verification	20%	2	4	4	4	5
Architectural complexity	20%	2	4	2	5	3
Feedback/adaptation	15%	2	3	5	4	5
Weighted score / 5	**100%**	**2.40**	**3.60**	**3.25**	**4.50**	**4.00**

Weighted score is the sum of weight multiplied by rating. Risk analysis receives the largest weight because correctness under failure is central to `kvstore`. The result supports Spiral as the primary model, with incremental delivery and Agile feedback practices.

9 Limitations and Assumptions

- This is an academic recommendation based on the supplied handout and SRS, not evidence that a formal Spiral process was historically followed.
- Ratings and weights are explicit judgement calls; stakeholders could choose different weights.
- The SRS records deliberate limits, including no general-purpose arbitrary cross-shard read transactions, classical statistical rather than trained ML techniques in operational intelligence, and leadership transfer that is not automatic by default. The process plan should preserve these boundaries unless requirements change formally.
- No implementation, performance, or safety guarantee is inferred from the model recommendation alone; such claims require test evidence and deployment-specific validation.

10 Result / Conclusion

The five models were compared and the four Lab 2 cases analysed. Waterfall is selected for the stable college payroll case, Agile for the evolving food-delivery app, Spiral for the high-risk hospital monitoring system, and Prototyping for the uncertain college portal interface.

For `kvstore`, the recommended primary model is **Spiral**, supported by incremental implementation, continuous testing, and Agile-style feedback. This combination best balances high technical risk, modular architecture, evolving subsystem details, and repeated verification under failure.

End of Experiment 2